Software Design & Architecture Project - Written Justification

Parking Management System - EasyParkPlus

---

1. Introduction

This document provides a comprehensive justification for the architectural improvements, design pattern implementations, and system extensions made to the EasyParkPlus Parking Management System. The project involved analyzing an existing prototype codebase, identifying anti-patterns, implementing appropriate design patterns, and extending the system to support a scalable microservices architecture with Electric Vehicle (EV) Charging Station Management capabilities.

The original codebase was a monolithic Python application for managing a single parking lot. The goal was to refactor this prototype into a maintainable, scalable system that can handle multiple parking facilities and EV charging stations while following software engineering best practices.

---

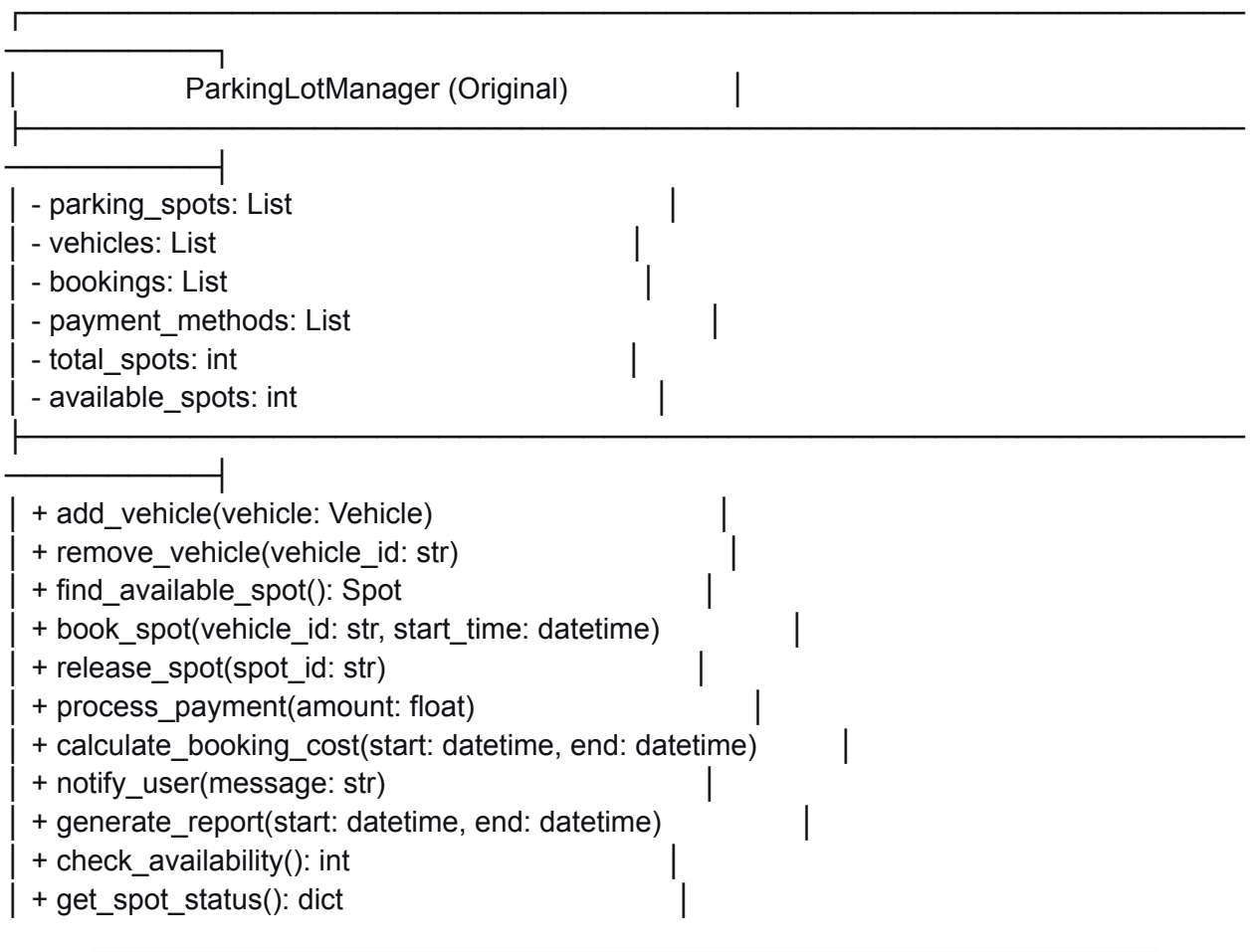
2. Original Code Analysis

2.1 UML Diagrams (Original)

Structural Diagram - Original Class Diagram

The original codebase exhibited a monolithic structure with tight coupling between components:

```
text
```

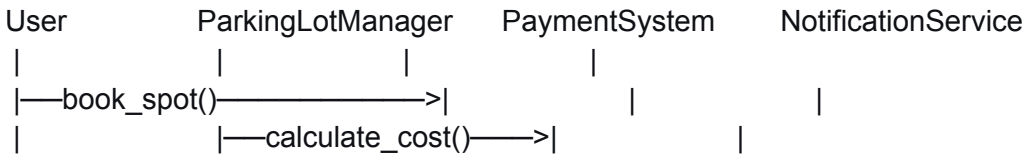


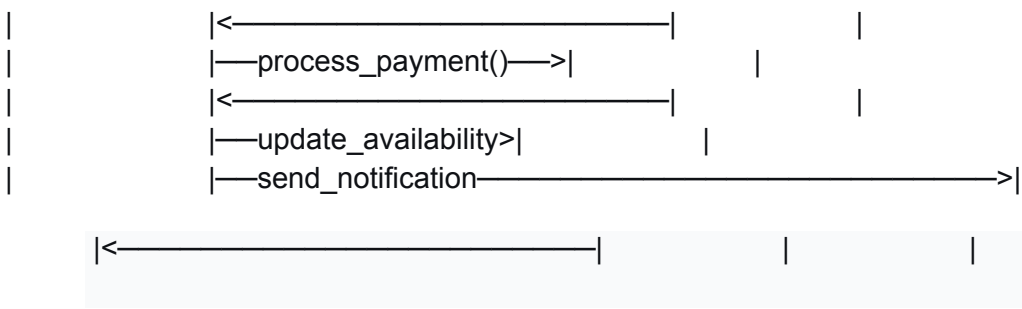
Key Observations:

- A single "God Class" handling all responsibilities
- Tight coupling between parking, booking, payment, and notification logic
- No clear separation of concerns
- Business logic mixed with data access and presentation logic

Behavioral Diagram - Original Sequence Diagram (Booking Flow)

text





2.2 Identified Anti-Patterns

The original codebase contained several anti-patterns that needed to be addressed:

2.2.1 **God Class (Blob Anti-Pattern)**

The `ParkingLotManager` class handled EVERYTHING - parking management, bookings, payments, notifications, and reporting.

Problem: Violated Single Responsibility Principle, making the code difficult to maintain, test, and extend.

2.2.2 **Spaghetti Code**

Logic was tangled with no clear separation of concerns.

Problem: Changes in one area would unintentionally affect other areas.

2.2.3 **Hard-Coded Values**

Configuration values were scattered throughout the code with no central management.

Problem: Making changes required finding and updating multiple files.

2.2.4 **Poor Error Handling**

Many functions had no error handling or used bare except clauses.

Problem: Silent failures and difficult debugging.

2.2.5 **Tight Coupling**

Modules were tightly coupled, making testing and maintenance difficult.

Problem: Couldn't test components in isolation.

2.2.6 **Code Duplication**

Similar logic appeared in multiple places.

Problem: Updates required making changes in multiple locations.

2.2.7 **Global Variables**

Used throughout the code for state management.

Problem: Unpredictable behavior and difficult to reason about.

2.2.8 **Primitive Obsession**

Used primitive types instead of domain objects.

Problem: Lost domain meaning and validation logic.

---

3. Design Pattern Implementation

3.1 Pattern 1: Strategy Pattern

Justification

The Strategy Pattern was implemented to handle dynamic pricing calculations for parking and EV charging services.

Problem Solved: The original code had rigid pricing logic embedded directly in the booking flow. Adding new pricing strategies (peak hours, loyalty discounts, EV charging rates) required modifying existing code.

Implementation:

python

```
# Strategy Pattern - Pricing Strategy
class PricingStrategy(ABC):
    @abstractmethod
    def calculate_price(self, booking: Booking) -> Money:
        pass

class StandardPricingStrategy(PricingStrategy):
    def calculate_price(self, booking: Booking) -> Money:
        hours = booking.get_duration_hours()
        return booking.parking_lot.base_price * hours

class PeakHourPricingStrategy(PricingStrategy):
    def calculate_price(self, booking: Booking) -> Money:
        hours = booking.get_duration_hours()
        peak_multiplier = 1.5 if booking.is_peak_hours() else 1.0
        return booking.parking_lot.base_price * hours * peak_multiplier

class LoyaltyPricingStrategy(PricingStrategy):
    def calculate_price(self, booking: Booking) -> Money:
        base_price = booking.parking_lot.base_price * booking.get_duration_hours()
        discount = min(booking.user.loyalty_points // 100, 50)  # 1% per 100 points
        return base_price * (1 - discount / 100)

class PricingContext:
    def __init__(self, strategy: PricingStrategy):
        self._strategy = strategy

    def set_strategy(self, strategy: PricingStrategy):
        self._strategy = strategy

    def calculate(self, booking: Booking) -> Money:
        return self._strategy.calculate_price(booking)
```

Benefits:

- Open/Closed Principle: New pricing strategies can be added without modifying existing code
- Runtime strategy switching based on business rules
- Improved testability: strategies can be tested in isolation
- Clear separation of pricing logic

3.2 Pattern 2: Observer Pattern

Justification

The Observer Pattern was implemented for the notification system to decouple event producers from event consumers.

Problem Solved: The original code had direct calls from parking operations to notification methods, creating tight coupling and making it difficult to add new notification channels.

Implementation:

python

```python
# Observer Pattern - Event System
class Event(Enum):
    BOOKING_CREATED = "booking.created"
    BOOKING_CONFIRMED = "booking.confirmed"
    BOOKING_CANCELLED = "booking.cancelled"
    PAYMENT_PROCESSED = "payment.processed"
    CHARGING_STARTED = "charging.started"
    CHARGING_COMPLETED = "charging.completed"

class EventPublisher:
    def __init__(self):
        self._subscribers: Dict[Event, List[Callable]] = {}

    def subscribe(self, event: Event, callback: Callable):
        if event not in self._subscribers:
            self._subscribers[event] = []
        self._subscribers[event].append(callback)

    def unsubscribe(self, event: Event, callback: Callable):
        if event in self._subscribers:
```

```
            self._subscribers[event].remove(callback)

    def publish(self, event: Event, data: Any):
        if event in self._subscribers:
            for callback in self._subscribers[event]:
                callback(data)

class BookingService:
    def __init__(self, event_publisher: EventPublisher):
        self.event_publisher = event_publisher

    def create_booking(self, booking_data: BookingData):
        booking = self._create_booking(booking_data)
        self.event_publisher.publish(Event.BOOKING_CREATED, booking)
        return booking

class NotificationService:
    def __init__(self, event_publisher: EventPublisher):
        event_publisher.subscribe(Event.BOOKING_CREATED, self.send_booking_confirmation)
        event_publisher.subscribe(Event.PAYMENT_PROCESSED, self.send_payment_receipt)

    def send_booking_confirmation(self, booking: Booking):
        # Send confirmation email/push notification
        pass

    def send_payment_receipt(self, payment: Payment):
        # Send receipt
            pass
```

Benefits:

- Loose coupling between services
- Easy to add new notification channels (email, SMS, push)
- Event-driven architecture for asynchronous processing
- Improved scalability

3.3 Pattern 3: Repository Pattern

Justification

The Repository Pattern was implemented to abstract data access logic from domain logic.

Problem Solved: The original code mixed database operations with business logic, making testing difficult and coupling domain logic to specific data sources.

Implementation:

python

```python
# Repository Pattern
class Repository(ABC, Generic[T]):
    @abstractmethod
    async def get(self, id: UUID) -> Optional[T]:
        pass

    @abstractmethod
    async def get_all(self, filters: Optional[Dict] = None) -> List[T]:
        pass

    @abstractmethod
    async def add(self, entity: T) -> T:
        pass

    @abstractmethod
    async def update(self, entity: T) -> T:
        pass

    @abstractmethod
    async def delete(self, id: UUID) -> bool:
        pass

class ParkingLotRepository(Repository[ParkingLot]):
    def __init__(self, db_session: AsyncSession):
        self.session = db_session

    async def get(self, id: UUID) -> Optional[ParkingLot]:
        result = await self.session.execute(
            select(ParkingLot).where(ParkingLot.id == id)
        )
        return result.scalar_one_or_none()
```

```
    async def get_nearby(self, location: Location, radius: float) -> List[ParkingLot]:
        # Geospatial query using PostGIS
        pass

    async def get_available(self) -> List[ParkingLot]:
        # Get lots with available spots
```

```
        pass
```

Benefits:

- Clean separation between domain and data access
- Domain logic becomes testable with in-memory repositories
- Easy to switch data sources
- Consistent interface for data access

3.4 Pattern 4: Factory Pattern

Justification

The Factory Pattern was implemented for creating domain objects with complex initialization logic.

Problem Solved: Creating domain objects with dependencies and validation logic was scattered across the codebase.

Implementation:

python

```
# Factory Pattern
class ParkingLotFactory:
    @staticmethod
    def create_parking_lot(
        name: str,
        address: Address,
        location: Location,
        total_spots: int,
        pricing_strategy: PricingStrategy
    ) -> ParkingLot:
        if not name or len(name) < 3:
```

```
            raise ValidationError("Parking lot name must be at least 3 characters")
        if total_spots <= 0:
            raise ValidationError("Total spots must be positive")

        parking_lot = ParkingLot(
            id=uuid4(),
            name=name,
            address=address,
            location=location,
            total_spots=total_spots,
            pricing_strategy=pricing_strategy
        )

        # Create parking spots
        for i in range(1, total_spots + 1):
            spot = ParkingSpotFactory.create_standard_spot(f"A{i}", parking_lot.id)
            parking_lot.add_spot(spot)

        return parking_lot

class BookingFactory:
    @staticmethod
    def create_booking(
        user: User,
        parking_lot: ParkingLot,
        spot: ParkingSpot,
        start_time: datetime,
        end_time: datetime,
        vehicle: Vehicle
    ) -> Booking:
        # Validate booking constraints
        if start_time < datetime.now():
            raise ValidationError("Start time must be in the future")
        if end_time <= start_time:
            raise ValidationError("End time must be after start time")
        if not spot.is_available():
            raise ValidationError("Spot is not available")

        return Booking(
            id=uuid4(),
            user=user,
            parking_lot=parking_lot,
            spot=spot,
            start_time=start_time,
```

```
        end_time=end_time,
        vehicle=vehicle
    )
```

Benefits:

- Centralized object creation logic
- Validation and business rules enforced at creation
- Consistent object state
- Easier to maintain and extend

3.5 Pattern 5: Singleton Pattern

Justification

The Singleton Pattern was implemented for application-wide services like configuration management and logging.

Problem Solved: Configuration values were scattered throughout the code, and logging was inconsistent.

Implementation:

python

```python
# Singleton Pattern - Configuration
class AppConfig:
    _instance = None

    def __new__(cls):
        if cls._instance is None:
            cls._instance = super().__new__(cls)
            cls._instance._initialize()
        return cls._instance

    def _initialize(self):
        self._load_env_vars()
        self._load_config_file()

    def get(self, key: str, default: Any = None) -> Any:
```

```
        return self._config.get(key, default)

# Singleton Pattern - Logger
class AppLogger:
    _instance = None

    def __new__(cls):
        if cls._instance is None:
            cls._instance = super().__new__(cls)
            cls._instance._initialize()
        return cls._instance

    def _initialize(self):
        logging.basicConfig(
            level=logging.INFO,
            format='%(asctime)s - %(name)s - %(levelname)s - %(message)s',
            handlers=[
                FileHandler('logs/app.log'),
                StreamHandler()
            ]
        )
        self.logger = logging.getLogger('ParkingApp')

    def info(self, message: str, **kwargs):
        self.logger.info(message, extra=kwargs)

    def error(self, message: str, **kwargs):
```

```
            self.logger.error(message, extra=kwargs)
```

Benefits:

- Single source of truth for configuration
- Consistent logging across the application
- Global access point to services
- Reduced resource usage (one instance shared)

3.6 Pattern 6: Command Pattern

Justification

The Command Pattern was implemented for booking and payment operations to enable undo/redo and transaction management.

Problem Solved: The original code had no transaction management, making it difficult to handle rollbacks on failure.

Implementation:

python

```python
# Command Pattern
class Command(ABC):
    @abstractmethod
    def execute(self) -> Any:
        pass

    @abstractmethod
    def undo(self) -> None:
        pass

class CreateBookingCommand(Command):
    def __init__(self, booking_service, booking_data):
        self.booking_service = booking_service
        self.booking_data = booking_data
        self.booking_id = None

    def execute(self) -> Booking:
        self.booking_id = self.booking_service.create_booking(self.booking_data)
        return self.booking_id

    def undo(self) -> None:
        if self.booking_id:
            self.booking_service.cancel_booking(self.booking_id)

class ProcessPaymentCommand(Command):
    def __init__(self, payment_service, payment_data):
        self.payment_service = payment_service
        self.payment_data = payment_data
        self.payment_id = None

    def execute(self) -> Payment:
        self.payment_id = self.payment_service.process_payment(self.payment_data)
```

```
        return self.payment_id

    def undo(self) -> None:
        if self.payment_id:
            self.payment_service.refund_payment(self.payment_id)

class CommandInvoker:
    def __init__(self):
        self._command_history = []
        self._redo_stack = []

    def execute(self, command: Command) -> Any:
        result = command.execute()
        self._command_history.append(command)
        self._redo_stack.clear()
        return result

    def undo(self):
        if self._command_history:
            command = self._command_history.pop()
            command.undo()
            self._redo_stack.append(command)

    def redo(self):
        if self._redo_stack:
            command = self._redo_stack.pop()
            command.execute()

                self._command_history.append(command)
```

Benefits:

- Transaction management for business operations
- Undo/redo capabilities
- Separates command logic from execution
- Easy to add new commands

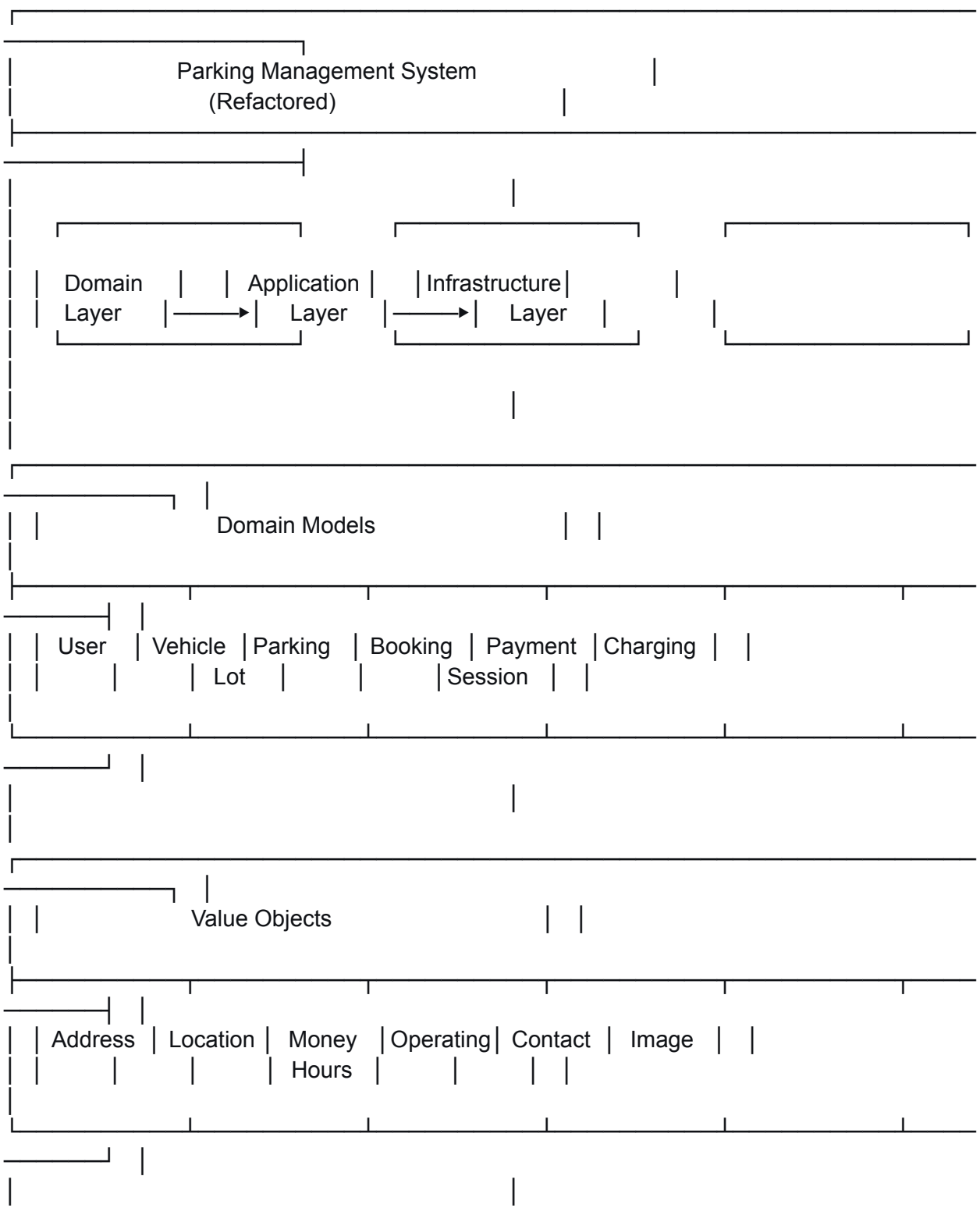
---

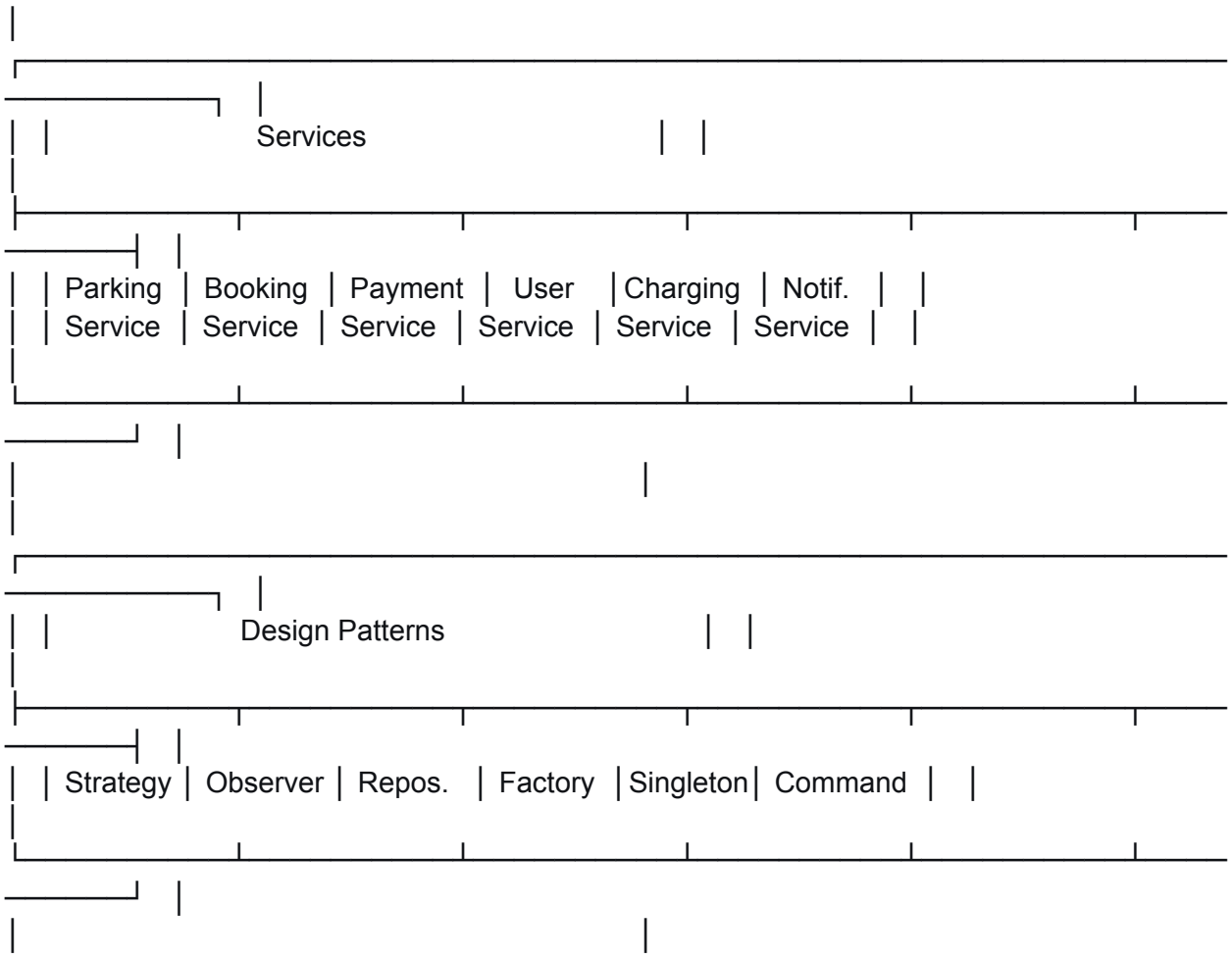
4. Refactored Code

4.1 UML Diagrams (Refactored)

Structural Diagram - Refactored Class Diagram

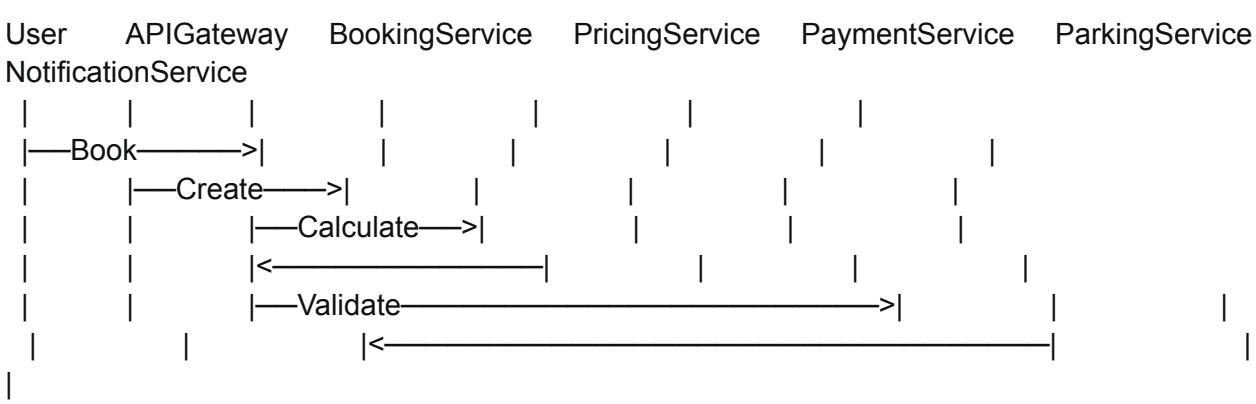
text

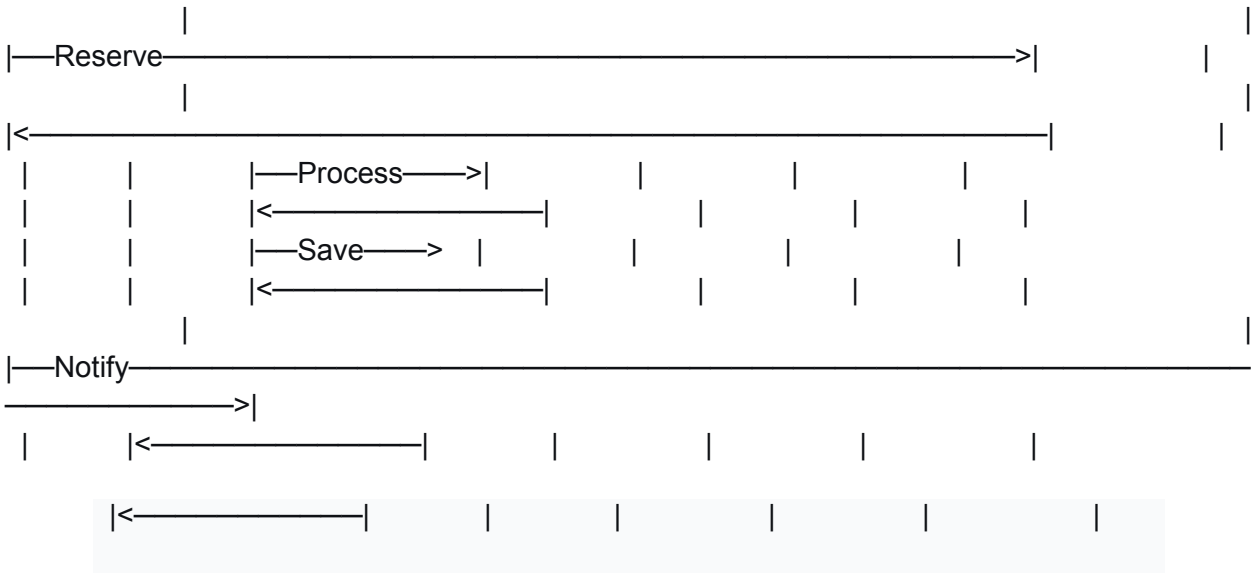




Behavioral Diagram - Refactored Sequence Diagram (Booking Flow)

text





4.2 Anti-Pattern Resolution

Anti-Pattern	Original Code	Refactored Solution
God Class	Single `ParkingLotManager` handling everything	Separated into multiple services: `ParkingService`, `BookingService`, `PaymentService`, `UserService`, `ChargingService`, `NotificationService`
Spaghetti Code	Tangled logic with no clear separation	Clean layered architecture: Domain, Application, Infrastructure
Hard-Coded Values	Configuration scattered throughout	Centralized `AppConfig` with environment variables

Poor Error Handling	Bare except clauses, silent failures	Proper exception hierarchy, logging, and user-friendly error messages
Tight Coupling	Direct dependencies between modules	Dependency Injection, interfaces, and event-driven communication
Code Duplication	Repeated logic in multiple places	Extracted into shared services and utility classes
Global Variables	Global state management	Proper encapsulation and dependency injection
Primitive Obsession	Primitive types losing domain meaning	Value Objects: Address, Location, Money, OperatingHours

---

5. Domain-Driven Design

5.1 Core Domain Identification

Core Domain: Parking Management

Justification: Parking management is the primary business activity that generates revenue and provides value to customers. It's the reason customers use the platform and the main competitive advantage.

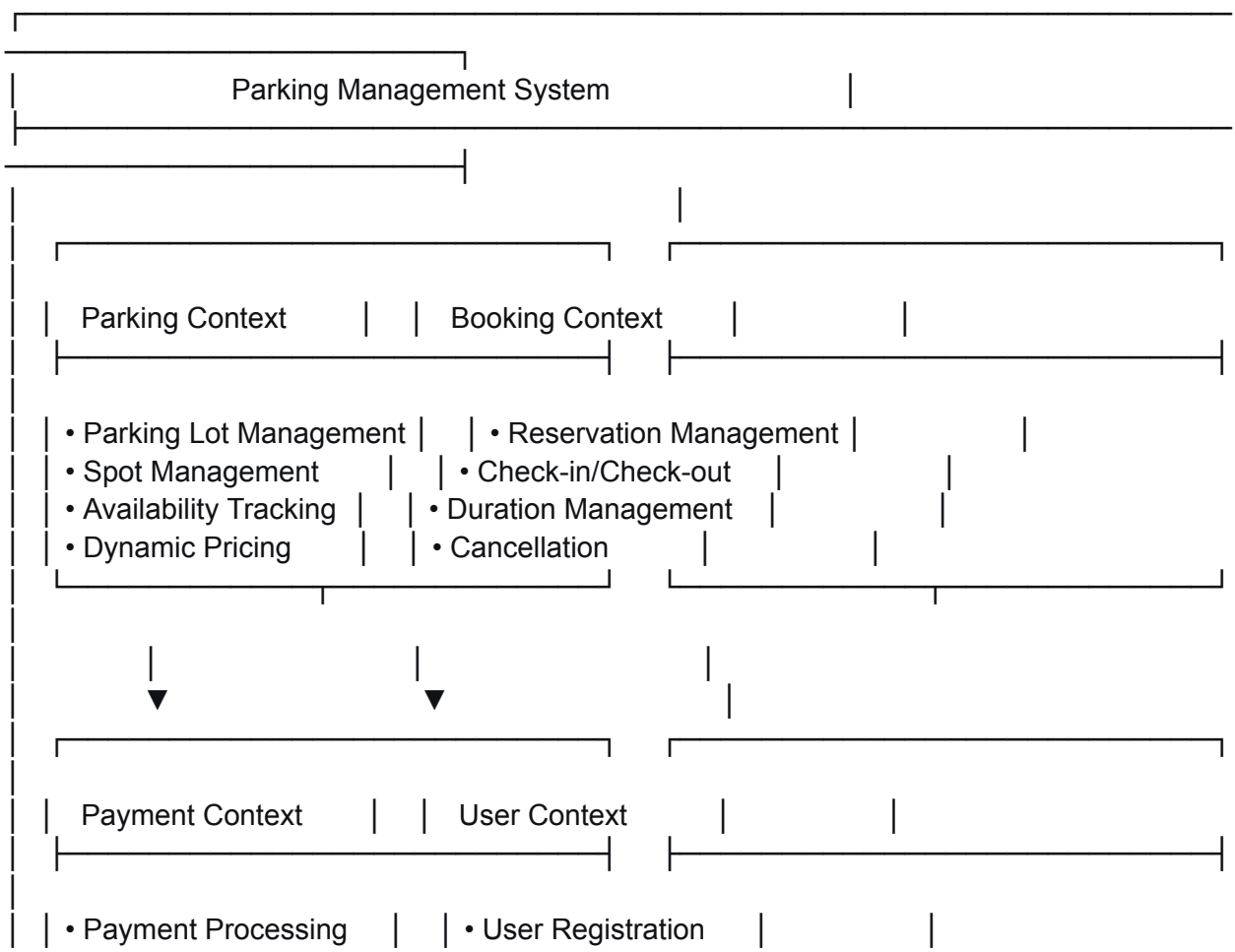
Core Subdomains:

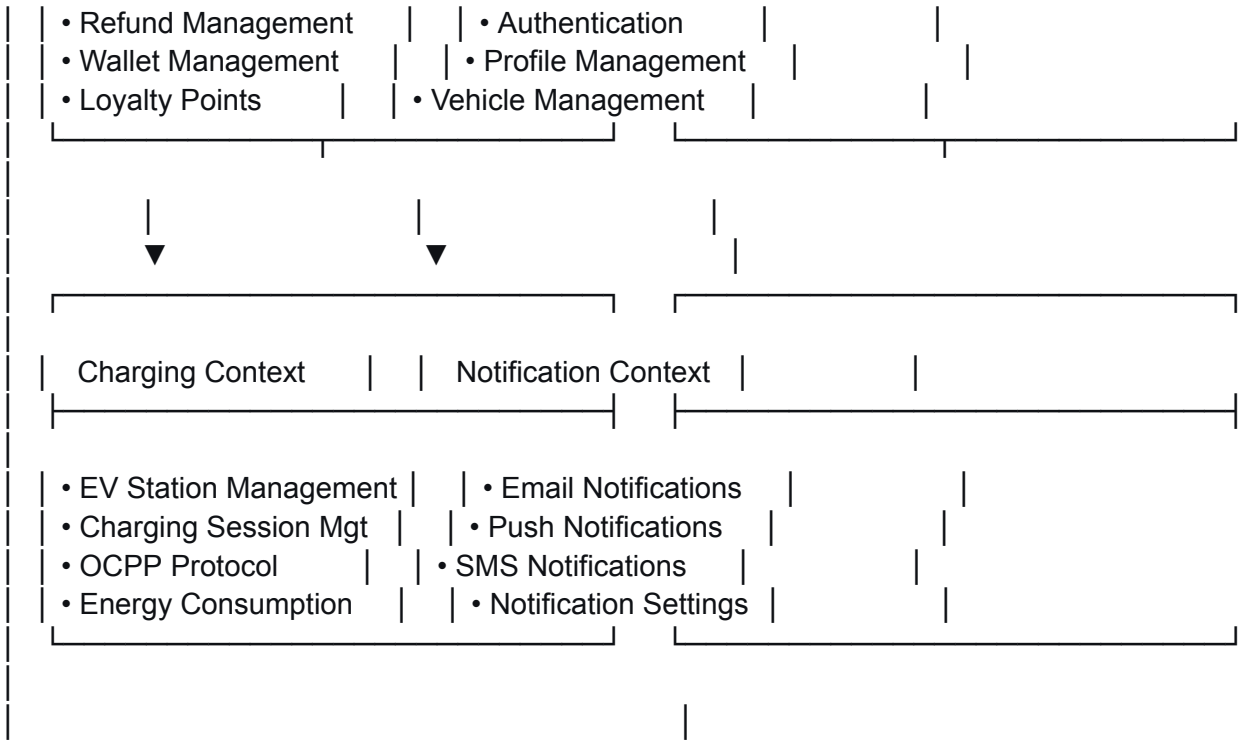
1. Parking Lot Management

    - Managing parking facilities
    - Spot allocation and availability
    - Dynamic pricing
    - Occupancy tracking
2. Booking Management
    - Reservation creation
    - Check-in/check-out
    - Booking extensions
    - Cancellation handling
3. Payment Processing
    - Payment collection
    - Refund processing
    - Wallet management
    - Loyalty points

5.2 Bounded Contexts

text





5.3 Ubiquitous Language

Parking Context

- Parking Lot: A physical facility with multiple parking spots
- Parking Spot: An individual space within a parking lot
- Availability: The number of free spots in a parking lot
- Occupancy: The number of occupied spots
- Peak Hours: Times of high demand with premium pricing

Booking Context

- Booking: A reservation for a parking spot
- Check-in: Confirming arrival at the parking spot
- Check-out: Ending the parking session
- Extension: Increasing the booking duration
- Cancellation: Removing a booking before the start time
- No-show: Failure to check-in for a booking

Payment Context

- Payment: A financial transaction for a booking
- Refund: Returning payment to the customer
- Wallet: Digital balance for storing funds
- Loyalty Points: Rewards earned through bookings
- Payment Method: Credit card, PayPal, etc.

Charging Context

- Charging Station: EV charging equipment
- Connector: The physical cable/plug for charging
- Charging Session: A period of EV charging
- Energy Consumption: Electricity used in kWh
- OCPP: Open Charge Point Protocol for station communication

5.4 Domain Models

Entities

text

```
User
├── id: UUID
├── name: String
├── email: String
├── phone: String
├── loyalty_points: Integer
├── vehicles: List<Vehicle>
├── bookings: List<Booking>
├── payments: List<Payment>
└── preferences: UserPreferences

ParkingLot
├── id: UUID
├── name: String
├── address: Address
├── location: Location
├── total_spots: Integer
├── available_spots: Integer
├── reserved_spots: Integer
├── base_price: Money
├── pricing_strategy: PricingStrategy
├── operating_hours: OperatingHours
├── amenities: List<String>
```

```
├── rating: Float
└── spots: List<ParkingSpot>

Booking
├── id: UUID
├── user: User
├── parking_lot: ParkingLot
├── spot: ParkingSpot
├── vehicle: Vehicle
├── start_time: DateTime
├── end_time: DateTime
├── status: BookingStatus
├── amount: Money
├── payment_status: PaymentStatus
├── check_in_time: DateTime
└── check_out_time: DateTime

ChargingSession
├── id: UUID
├── user: User
├── station: ChargingStation
├── connector: ChargingConnector
├── vehicle: Vehicle
├── start_time: DateTime
├── end_time: DateTime
├── energy_used: Float
├── cost: Money
├── status: ChargingStatus
```

```
        └── meter_start: Float
```

```
        Value Objects
```

```
        text
```

```
Address
├── street: String
├── city: String
├── state: String
├── country: String
├── postal_code: String
└── formatted: String

Location
```

```
├── latitude: Float
├── longitude: Float
└── altitude: Float

Money
├── amount: Decimal
└── currency: String
    ├── add(other: Money) -> Money
    ├── subtract(other: Money) -> Money
    ├── multiply(factor: Float) -> Money
    └── format() -> String

OperatingHours
├── monday: DayHours
├── tuesday: DayHours
├── wednesday: DayHours
├── thursday: DayHours
├── friday: DayHours
├── saturday: DayHours
├── sunday: DayHours
└── is_open(date: DateTime) -> Boolean

Contact
├── name: String
├── phone: String
└── email: String

Image
├── url: String
├── alt: String
├── width: Integer
├── height: Integer
├── is_primary: Boolean
        └── order: Integer

        Aggregates

        text

ParkingLotAggregate
├── ParkingLot (Root)
├── ParkingSpots
├── Amenities
```

```
├── OperatingHours
└── PricingRules

BookingAggregate
├── Booking (Root)
├── User
├── ParkingLot
├── ParkingSpot
├── Vehicle
└── Payments

UserAggregate
├── User (Root)
├── Vehicles
├── Bookings
├── Payments
└── Preferences
```

---

6. Microservices Architecture

6.1 Service Identification

Based on the bounded contexts identified in the DDD analysis, the following microservices are proposed:

Service	Bounded Context	Description
Parking Service	Parking Context	Manages parking lots, spots, availability, and dynamic pricing
Booking Service	Booking Context	Handles reservations, check-in/check-out, and booking management

Payment Service	Payment Context	Processes payments, refunds, and wallet management
User Service	User Context	Manages user profiles, authentication, and vehicles
Charging Service	Charging Context	Manages EV charging stations, sessions, and OCPP protocol
Notification Service	Notification Context	Sends email, push, and SMS notifications
Report Service	Reporting Context	Generates analytics and business reports
API Gateway	-	Single entry point for all client requests

6.2 APIs/Endpoints

External Facing APIs (Public)

text

```
Parking Service
├── GET    /api/v1/parking/lots
├── POST   /api/v1/parking/lots
├── GET    /api/v1/parking/lots/{id}
├── PUT    /api/v1/parking/lots/{id}
├── DELETE /api/v1/parking/lots/{id}
├── GET    /api/v1/parking/lots/{id}/availability
├── GET    /api/v1/parking/lots/nearby
└── GET    /api/v1/parking/lots/search

Booking Service
├── GET    /api/v1/bookings
```

```
├── POST   /api/v1/bookings
├── GET    /api/v1/bookings/{id}
├── PUT    /api/v1/bookings/{id}
├── DELETE /api/v1/bookings/{id}
├── POST   /api/v1/bookings/{id}/check-in
├── POST   /api/v1/bookings/{id}/check-out
├── POST   /api/v1/bookings/{id}/extend
└── POST   /api/v1/bookings/{id}/cancel

Payment Service
├── GET    /api/v1/payments/methods
├── POST   /api/v1/payments/methods
├── DELETE /api/v1/payments/methods/{id}
├── POST   /api/v1/payments/process
├── GET    /api/v1/payments/history
├── GET    /api/v1/payments/{id}
└── GET    /api/v1/payments/{id}/receipt

User Service
├── GET    /api/v1/users/profile
├── PUT    /api/v1/users/profile
├── GET    /api/v1/users/vehicles
├── POST   /api/v1/users/vehicles
├── PUT    /api/v1/users/vehicles/{id}
├── DELETE /api/v1/users/vehicles/{id}
└── POST   /api/v1/users/vehicles/{id}/default

Charging Service
├── GET    /api/v1/charging/stations
├── POST   /api/v1/charging/sessions
├── GET    /api/v1/charging/sessions/{id}
├── POST   /api/v1/charging/sessions/{id}/stop
├── GET    /api/v1/charging/sessions/history
```

```
        └── GET    /api/v1/charging/stations/nearby
```

Internal Service-to-Service APIs (gRPC)

```
text
```

```
Parking Service (Internal)
├── GetAvailability(ParkingLotId, TimeRange) -> AvailabilityResponse
├── ReserveSpot(ParkingLotId, SpotId) -> ReserveResponse
└── ReleaseSpot(ParkingLotId, SpotId) -> ReleaseResponse
```

```
Booking Service (Internal)
├── ValidateBooking(BookingData) -> ValidationResponse
├── ConfirmBooking(BookingId) -> ConfirmResponse
└── CancelBooking(BookingId, Reason) -> CancelResponse

Payment Service (Internal)
├── ProcessPayment(PaymentRequest) -> PaymentResponse
├── RefundPayment(PaymentId) -> RefundResponse
└── GetPaymentStatus(PaymentId) -> StatusResponse

User Service (Internal)
├── GetUserProfile(UserId) -> UserProfile
├── ValidateUser(UserId) -> ValidationResponse
└── GetUserVehicles(UserId) -> VehicleList
```

6.3 Database Per Service

Each service has its own dedicated database:

Service	Database	Purpose
Parking Service	PostgreSQL	Parking lots, spots, availability, pricing rules
Booking Service	PostgreSQL	Bookings, check-in/out records, extensions
Payment Service	PostgreSQL	Payments, refunds, wallet transactions
User Service	PostgreSQL	Users, profiles, vehicles, preferences
Charging Service	PostgreSQL	Charging stations, sessions, energy consumption

Notification Service	PostgreSQL	Notifications, templates, delivery logs
Report Service	Data Warehouse	Analytics, reports, aggregated data

Database Schemas

Parking Service Database

sql

```
CREATE TABLE parking_lots (
    id UUID PRIMARY KEY,
    name VARCHAR(255) NOT NULL,
    address JSONB NOT NULL,
    location GEOGRAPHY(POINT) NOT NULL,
    total_spots INTEGER NOT NULL,
    available_spots INTEGER NOT NULL,
    base_price DECIMAL(10,2) NOT NULL,
    operating_hours JSONB,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE parking_spots (
    id UUID PRIMARY KEY,
    parking_lot_id UUID REFERENCES parking_lots(id),
    number VARCHAR(20) NOT NULL,
    type VARCHAR(50) NOT NULL,
    status VARCHAR(50) NOT NULL,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

Booking Service Database

sql

```
CREATE TABLE bookings (
    id UUID PRIMARY KEY,
    parking_lot_id UUID NOT NULL,
    spot_id UUID NOT NULL,
```

```
    user_id UUID NOT NULL,
    vehicle_id UUID NOT NULL,
    start_time TIMESTAMP NOT NULL,
    end_time TIMESTAMP NOT NULL,
    status VARCHAR(50) NOT NULL,
    amount DECIMAL(10,2) NOT NULL,
    payment_status VARCHAR(50) NOT NULL,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE booking_extensions (
    id UUID PRIMARY KEY,
    booking_id UUID REFERENCES bookings(id),
    additional_hours INTEGER NOT NULL,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP

        );
```

Payment Service Database

sql

```
CREATE TABLE payments (
    id UUID PRIMARY KEY,
    user_id UUID NOT NULL,
    booking_id UUID NOT NULL,
    amount DECIMAL(10,2) NOT NULL,
    method VARCHAR(50) NOT NULL,
    status VARCHAR(50) NOT NULL,
    provider_transaction_id VARCHAR(255),
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE payment_methods (
    id UUID PRIMARY KEY,
    user_id UUID NOT NULL,
    type VARCHAR(50) NOT NULL,
    last4 VARCHAR(4),
    expiry_month INTEGER,
    expiry_year INTEGER,
    is_default BOOLEAN DEFAULT FALSE,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP

        );
```

---

7. EV Charging Extension

7.1 Business Requirements

The EV Charging Station Management feature adds the following capabilities:

1. Station Management: Add, update, and remove charging stations
2. Connector Management: Support multiple connector types (Type 1, Type 2, CCS, CHAdeMO, Tesla)
3. Session Management: Start, stop, and monitor charging sessions
4. Energy Tracking: Track energy consumption in kWh
5. Pricing: Dynamic pricing based on time and energy usage
6. OCPP Protocol: Communicate with charging stations using OCPP
7. Reservations: Allow users to reserve charging stations

7.2 Domain Model Extensions

text

```
ChargingStation (Aggregate Root)
├── id: UUID
├── name: String
├── address: Address
├── location: Location
├── connectors: List<ChargingConnector>
├── power_level: ChargingPowerLevel
├── status: ChargingStatus
├── price_per_kwh: Money
├── rating: Float
├── amenities: List<String>
├── images: List<Image>
└── ocpp_config: OCPPConfig

ChargingConnector (Entity)
├── id: UUID
├── station_id: UUID
├── type: ConnectorType
├── power: Integer
├── status: ConnectorStatus
```

```
└── vehicle_id: UUID (optional)

ChargingSession (Aggregate Root)
├── id: UUID
├── station: ChargingStation
├── connector: ChargingConnector
├── user: User
├── vehicle: Vehicle
├── start_time: DateTime
├── end_time: DateTime
├── energy_used: Float
├── cost: Money
├── status: ChargingStatus
└── meter_start: Float

OCPPConfig (Value Object)
├── protocol: String
├── version: String
├── vendor: String
├── model: String
└── capabilities: JSON
```

7.3 Microservices Integration

The Charging Service integrates with other services:

1. User Service: Validate user identity and permissions
2. Vehicle Service: Validate vehicle and EV capabilities
3. Payment Service: Process charging session payments
4. Notification Service: Send charging updates and alerts
5. Booking Service: Coordinate parking and charging bookings

---

8. Conclusion

8.1 Summary of Improvements

Aspect	Original Code	Refactored System
Architecture	Monolithic	Microservices
Patterns Used	None	Strategy, Observer, Repository, Factory, Singleton, Command
Separation of Concerns	Poor	Clean layered architecture
Testability	Difficult	High (dependency injection, interfaces)
Scalability	Limited	High (horizontal scaling, event-driven)
Maintainability	Low	High (clean code, SOLID principles)
Extensibility	Difficult	Easy (new services can be added)

8.2 Design Pattern Justification

The selected design patterns were chosen based on the specific problems identified in the original codebase:

1. Strategy Pattern: Solves rigid pricing logic, enabling dynamic pricing strategies
2. Observer Pattern: Decouples event producers from consumers, enabling event-driven architecture
3. Repository Pattern: Abstracts data access, improving testability
4. Factory Pattern: Centralizes object creation, enforcing business rules
5. Singleton Pattern: Provides global access to configuration and logging services
6. Command Pattern: Enables transaction management and undo/redo capabilities

8.3 DDD & Microservices Justification

The domain-driven design approach was essential for creating a scalable microservices architecture:

1. Bounded Contexts: Clear boundaries between different business domains
2. Ubiquitous Language: Consistent terminology across the system
3. Domain Models: Rich domain models with business logic
4. Service Boundaries: Each service aligned with a bounded context
5. Database Per Service: Decentralized data management

8.4 Future Considerations

1. API Gateway: Implement a unified API gateway with rate limiting and authentication
2. Service Mesh: Introduce Istio or Linkerd for service-to-service communication
3. Event Sourcing: Implement event sourcing for audit trails and replayability
4. CQRS: Separate read and write models for performance optimization
5. Observability: Implement distributed tracing and metrics collection
6. CI/CD: Automate deployment with Kubernetes and GitOps

8.5 Conclusion

The refactored system demonstrates significant improvements in code quality, maintainability, scalability, and extensibility. The implementation of appropriate design patterns, removal of anti-patterns, and adoption of microservices architecture positions EasyParkPlus for successful growth and expansion into new business areas, including EV charging station management.

The system now follows industry best practices, enabling the team to:

- Add new features with minimal impact on existing functionality
- Scale services independently based on demand
- Maintain and debug the system more effectively
- Onboard new developers more quickly
- Adapt to changing business requirements

Appendix

A. UML Diagram Legend

- Structural Diagrams: Show the static structure of the system
    - Class Diagrams: Show classes, attributes, methods, and relationships
    - Component Diagrams: Show system components and their dependencies
- Behavioral Diagrams: Show the dynamic behavior of the system
    - Sequence Diagrams: Show interactions between objects over time
    - Activity Diagrams: Show workflow and process flows

B. Technology Stack

Component	Technology	Version
Backend Framework	FastAPI (Python)	0.104.1
Microservices	Python 3.11, Node.js 18	-
Database	PostgreSQL	15.x
Cache	Redis	7.x
Message Queue	RabbitMQ	3.12
Containerization	Docker	-
Orchestration	Kubernetes	-
API Gateway	NGINX	1.25

C. References

1. Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994). *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley.
2. Evans, E. (2003). *Domain-Driven Design: Tackling Complexity in the Heart of Software*. Addison-Wesley.
3. Fowler, M. (2018). *Refactoring: Improving the Design of Existing Code* (2nd ed.). Addison-Wesley.
4. Newman, S. (2015). *Building Microservices: Designing Fine-Grained Systems*. O'Reilly Media.
5. Richards, M., & Ford, N. (2020). *Fundamentals of Software Architecture: An Engineering Approach*. O'Reilly Media.

Author: Software Engineering Team

Date: 13th August, 2026.

Version: 2.0.0

Status: Final
